

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Flood Situation Monitoring and Reporting System

By
Do Nguyen Chuong - ITITIU22021

*A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Computer Science*

Ho Chi Minh City, Vietnam
2026

Flood Situation Monitoring and Reporting System

APPROVED BY:

Le Duy Tan, Ph.D., supervisor

THESIS COMMITTEE

Le Duy Tan, Ph.D

.....

Acknowledgments

I would like to thank my thesis supervisor, Dr. Le Duy Tan, for his guidance throughout this work. His comments helped me narrow the topic, connect the system design with the actual source code, and keep the report focused when the project started to become too wide. I am especially grateful for the time he spent reviewing my ideas and pointing out where the work still needed to be clearer.

I also want to thank the lecturers at the School of Computer Science and Engineering. The courses and discussions I had there gave me the background I needed to build VietFlood across backend services, a mobile app, a web dashboard, and deployment tools.

I am thankful to the staff at International University as well. Their work behind the scenes made my study process smoother than it would have been otherwise.

My classmates and colleagues also helped more than they may realize. Some helped me talk through technical problems. Some simply listened when the project felt messy. Those small conversations made the work easier to continue.

Finally, I would like to thank my family for their patience and support during my studies. Their trust in me gave me room to finish this thesis.

This thesis carries the help of many people. I am grateful to all of them.

Table of Contents

List of Tables	vii
List of Figures	viii
List of Abbreviations	ix
Abstract	x
1 Introduction	1
1.1 Motivation and Problem	1
1.2 Research Objectives	1
1.3 Scope and Limitations	2
1.4 System Overview	2
1.5 Key Contributions	2
1.6 Chapter Roadmap	3
2 Background and Related Works	4
2.1 Flood Reporting as an Information Problem	4
2.1.1 Review States and Follow-Up	4
2.1.2 Mobile Reporting Constraints	4
2.2 Research Context and Key Concepts	6
2.2.1 Citizen Signals in Emergencies	6
2.2.2 Responsibility-Driven Architecture	6
2.3 Backend Decomposition Options	6
2.3.1 Gateway Layer and Service Messaging	6
2.4 Storage, Evidence, and Caching Choices	7
2.4.1 Relational Storage and JSON Evidence Metadata	7
2.4.2 External Evidence Storage (Cloudinary)	7
2.4.3 Redis Cache Layer	7
2.5 Access Control and Real-Time Channels	7
2.5.1 JWT Tokens and Password Hashing	7
2.5.2 Socket.IO for Live Location Updates	8
2.6 Deployment and Environment Repeatability	8
3 Methodology	9
3.1 Method Overview	9
3.2 Design Inputs and Constraints	9
3.2.1 Functional Goals	9
3.2.2 Quality and Operational Goals	10
3.3 Role and Use-Case Modeling	10
3.3.1 Citizen Role	10
3.3.2 Relief Role	10
3.3.3 Administrator Role	12
3.4 Development Materials and Tools	12
3.5 Architecture Design Approach	13

3.5.1	Gateway Interface Design	13
3.5.2	Service Messaging Design	14
3.6	Data Modeling Approach	14
3.6.1	Caching Approach	14
3.7	Workflow Specification	14
3.7.1	Evidence Metadata Handling	16
3.7.2	Review and Status Updates	16
3.8	Client Development Approach	16
3.8.1	Mobile Client Design	16
3.8.2	Dashboard Design	16
3.9	Real-Time Channel Approach	17
3.10	Prototype Validation Strategy	17
4	Implementation and Result	18
4.1	Prototype Deliverables and Technology Stack	18
4.2	Backend Implementation	19
4.2.1	Service Responsibilities and Routing	19
4.2.2	Authentication, Tokens, and Role Checks	19
4.2.3	Report Handling and Evidence Pipeline	21
4.2.4	Shared Infrastructure Utilities	21
4.2.5	Live Location Event Channel	22
4.3	Client-Side Implementation	22
4.3.1	Mobile Application Features	22
4.3.2	Administrator Dashboard Features	23
4.4	Deployment and Release Automation	25
4.5	Result Summary	25
5	Evaluation	27
5.1	Evaluation Approach	27
5.2	Workflow Coverage	27
5.3	Client Behavior Review	28
5.4	Storage and Data Consistency	28
5.5	Access Control and Security Considerations	28
5.6	Deployment Considerations	29
5.7	Summary of Findings	29
6	Conclusion and Future Works	30
6.1	Closing Summary	30
6.2	Recommended Next Steps	30
A	API Reference and Sample Payloads	34
A.1	Endpoint Inventory	34
A.2	Authentication: Sign In	34
A.3	Authentication: Token Refresh	35
A.4	User: Profile Payload	35
A.5	Reports: Create Payload	35
B	User Interface Snapshots	36
B.1	Mobile and Dashboard Screens	36

C	Prototype Validation Checklist	37
C.1	Auth and User Scenarios	37
C.2	Report and Evidence Scenarios	37
C.3	Tracking Scenarios (Socket.IO)	38
D	Runtime Configuration Summary	39
D.1	Operational Notes	39

List of Tables

3.1	Materials, tools, and runtime dependencies used during VietFlood development.	12
4.1	Implementation technologies used in the VietFlood prototype.	18
4.2	Responsibility split across backend services in VietFlood.	19
5.1	Observed coverage of key VietFlood workflows in the prototype.	28
A.1	REST endpoints exposed by the VietFlood API gateway.	34
C.1	Authentication and user management validation scenarios.	37
C.2	Report workflow and evidence handling validation scenarios.	37
C.3	Socket.IO live tracking validation scenarios.	38
D.1	Environment variable groups used for VietFlood deployment with Docker Compose.	39

List of Figures

2.1	VietFlood report flow from mobile submission to administrator review status.	5
2.2	High-level contrast between a monolithic backend and the VietFlood service split.	7
2.3	Example views of evidence upload on mobile and evidence review on the dashboard.	8
3.1	Citizen capabilities supported by VietFlood (use case view).	11
3.2	Relief browsing features supported by VietFlood (use case view).	11
3.3	Administrator review and management features (use case view).	12
3.4	System architecture overview of VietFlood components and their interactions.	13
3.5	Prototype database schema showing the core entities and relationships.	15
3.6	Core data structures used in the report workflow.	15
3.7	Sequence of steps from report submission to persistence and cache update.	16
4.1	RabbitMQ message routing between the API gateway and internal services.	19
4.2	Mobile registration view used to create a citizen account.	20
4.3	Mobile sign-in view that returns access and refresh tokens.	20
4.4	Administrator sign-in view in the web dashboard.	20
4.5	Refresh token rotation sequence for renewing access tokens and invalidating old refresh tokens.	20
4.6	End-to-end report handling pipeline from submission to persistence and cache refresh.	21
4.7	Structure of the shared utilities package used by backend services (logging, Redis, Cloudinary).	22
4.8	Socket.IO location event flow including validation, broadcast, and error responses.	22
4.9	Citizen report list showing submissions and current status values.	23
4.10	Report creation form used to enter description and location fields.	23
4.11	Evidence selection and upload workflow on the mobile client.	23
4.12	Relief list view for browsing broader report records.	23
4.13	Relief map view for locating reports spatially.	23
4.14	Administrator dashboard overview with report cards, filters, and evidence thumbnails.	24
4.15	Evidence review and status update screen used by administrators.	24
4.16	CI/CD pipeline from source push to container build, publication, and Azure App Service deployment.	25
4.17	User management view for browsing accounts and roles in the dashboard.	26
B.1	Mobile report creation screen (citizen workflow).	36
B.2	Mobile evidence selection and upload screen.	36
B.3	Relief report list screen for browsing incoming records.	36
B.4	Administrator dashboard overview with report list and filters.	36

List of Abbreviations

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
HTTP	Hypertext Transfer Protocol
UI/UX	User Interface / User Experience
CDN	Content Delivery Network
PUB/SUB	Publish–Subscribe
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
JWT	JSON Web Token
TTL	Time To Live
WS	WebSocket

Abstract

Flood response depends on small pieces of local information becoming useful at the right time. In many cases, the first sign of a flood problem is not an official record but a photo, a short description, or a shared location from a person already near the affected area. When those signals stay inside separate chat groups or informal channels, the information is hard to verify, search, and follow up. This thesis presents VietFlood, a prototype for a Flood Situation Monitoring and Reporting System that organizes citizen flood reports for relief awareness and administrator review.

VietFlood is implemented as a multi-platform prototype. The backend uses NestJS with an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. RabbitMQ supports service-to-service communication, PostgreSQL stores users, refresh tokens, and flood reports through TypeORM, Redis supports cached report lookup, and Cloudinary stores uploaded evidence files. Shared infrastructure code is kept in `vietflood_common`, which provides reusable logging, Redis, and Cloudinary helpers. The mobile application is built with Expo React Native for citizen and relief workflows. The web dashboard is built with Next.js, React, TypeScript, and Tailwind CSS for administrators.

The prototype covers the main workflow from report creation to review. Citizens can register, sign in, create flood reports, provide administrative address fields and coordinates, set urgency and severity, and upload photo evidence. Relief users can open broader report views and map-related screens. Administrators can use the web dashboard to inspect reporter information, search and filter reports, preview evidence, manage users, and update report status using pending, verified, resolved, and rejected states. VietFlood also includes a live tracking path in which connected clients send location updates through Socket.IO after latitude and longitude validation at the gateway.

The work focuses on system design and implementation rather than flood prediction or official emergency dispatch. The final prototype demonstrates how mobile reporting, role-based access control, evidence storage, real-time location events, shared backend utilities, and container-based deployment can be combined in a Vietnam-focused flood reporting system. Although it remains a prototype, VietFlood provides a working foundation for collecting field reports and making them easier to review.

Keywords: VietFlood; flood reporting; relief coordination; real-time tracking; microservice architecture; NestJS; React Native; Next.js; RabbitMQ; PostgreSQL; Redis; Cloudinary.

Chapter 1

Introduction

This chapter introduces the motivation for VietFlood and defines the problem addressed by the thesis. Flood events generate urgent, local information: photos of blocked roads, short messages about water level, and location pins shared between neighbors and volunteers. These signals can be valuable, but they are difficult to verify and reuse when they remain as isolated chat messages. VietFlood is designed to convert these observations into structured reports that can be stored, searched, reviewed, and tracked over time. This approach aligns with the broader idea of using citizen-generated crisis information as an input to disaster response when it is organized into usable data [1, 2].

1.1 Motivation and Problem

In real flood situations, the first useful information often comes from the field rather than from official channels. However, field reports are typically incomplete and inconsistent. A message may include a photo but no administrative address. A location pin may be shared without an explanation. Duplicate reports can describe the same incident in different wording. When information is fragmented like this, it becomes difficult for a reviewer to answer basic questions consistently: what happened, where it happened, who reported it, and whether the case has been verified or resolved.

The thesis problem can be summarized as follows: *how to provide a practical reporting system that turns ad-hoc flood observations into structured, reviewable records, while supporting different user roles and remaining simple enough to run as a prototype.* VietFlood addresses this problem by building a multi-platform prototype that supports report submission from mobile devices, evidence upload, persistent storage, role-based access control, administrator review, and status tracking.

1.2 Research Objectives

The main objective is to design and implement a working prototype for flood situation reporting and review. The prototype is required to demonstrate the end-to-end workflow rather than only a user interface or only a backend.

Specific objectives are:

1. Design a reporting data model with location fields, evidence metadata, timestamps, and a review status suitable for administrator follow-up.
2. Implement a backend that separates public API handling, authentication, and report management responsibilities using a small service split [3, 4, 5].
3. Support role-based access control using JWT authentication so citizens, relief users, and administrators see different capabilities [6].
4. Store reports and user accounts persistently using PostgreSQL and TypeORM, and store evidence metadata as structured JSON where appropriate [7, 8].

5. Store photo evidence outside the database using a media platform and keep references inside the report record [9].
6. Provide a mobile application for citizen reporting and relief-oriented browsing using Expo React Native [10, 11].
7. Provide an administrator web dashboard using Next.js to review reports, inspect evidence, and update report status [12].
8. Package the system so it can run in a repeatable environment using Docker Compose and be deployable using CI/CD practices [13, 14, 15, 16, 17].

1.3 Scope and Limitations

VietFlood is a prototype for information collection and review. It does not provide flood prediction, hydrological modeling, or automated dispatch. The prototype assumes that users can access the mobile app and that evidence uploads may be possible, but it also acknowledges that connectivity can be unreliable during floods. The system therefore prioritizes a clear data model, a role-based review workflow, and an architecture that can be tested locally and extended later.

1.4 System Overview

VietFlood consists of three user-facing components and a backend:

- **Mobile application:** a citizen and relief client built with Expo React Native. Citizens can authenticate, submit reports with location and optional evidence, and view personal report history. Relief users can browse broader report information and map-related views [10, 11].
- **Web dashboard:** an administrator interface built with Next.js that supports report review, evidence inspection, filtering, and status updates [12].
- **Backend services:** a NestJS-based backend that exposes a single public API surface via an API gateway and uses internal services for authentication and report management [18, 19].
- **Supporting infrastructure:** RabbitMQ for service messaging, Redis for caching, PostgreSQL for persistence, and Cloudinary for evidence storage [20, 21, 8, 9].

The key design decision is that a report is treated as a durable record. It is not only a message. Each report includes a description, location fields and coordinates, evidence references, and a status that can be updated by administrators after review.

1.5 Key Contributions

The contributions of this thesis are:

- A clear, Vietnam-focused reporting workflow that separates citizen submission, relief browsing, and administrator verification into distinct roles.

- A working multi-platform prototype (mobile + web + backend) that demonstrates report creation, evidence upload, persistent storage, and status tracking end-to-end.
- A service-oriented backend structure that keeps authentication and report handling responsibilities separated while remaining small enough for a thesis prototype.
- A documented API surface and evaluation checklist, supported by Appendix materials that list endpoints, response samples, and scenario-based test cases.

1.6 Chapter Roadmap

The thesis is organized as follows. Chapter 2 introduces background and related work on crisis information, structured reporting, and the architectural patterns used by VietFlood. Chapter 3 presents the methodology and the design process, including actor modeling, system structure, and data model choices. Chapter 4 describes the prototype implementation across backend services, the mobile application, and the administrator dashboard. Chapter 5 evaluates the prototype through workflow walkthroughs and discusses limitations. Chapter 6 concludes the thesis and outlines future improvements.

Chapter 2

Background and Related Works

2.1 Flood Reporting as an Information Problem

Flood reporting is an information problem before it is a software problem. The earliest signals of a flooded road or a blocked route usually come from people already near the affected area. These field observations arrive quickly, but they arrive in formats that are hard to reuse: a photo without a clear address, a short message without coordinates, or a location pin without enough context to judge severity. As a result, valuable information can be lost in message streams or become difficult to verify and track over time.

VietFlood follows the idea that citizen-generated information can support crisis response when it is organized into usable data. Research on crowdsourcing and crisis information highlights that local observations can provide timely coverage, but they require structure and processing to become useful in practice [1, 2]. In this thesis, a “report” is treated as a durable record with fields that support review: what happened, where it happened, who reported it, what evidence exists, and what the current status is.

2.1.1 Review States and Follow-Up

The simplest trust signal in a reporting platform is the review status. VietFlood uses a small status set suitable for a prototype: **pending**, **verified**, **resolved**, and **rejected**. A new submission is pending. An administrator can mark it verified after review, resolved when it is no longer active, or rejected when the report is incorrect or unsuitable. Keeping rejected reports separate supports accountability by preserving the record of what was submitted rather than silently discarding it.

Figure 2.1 summarizes the report lifecycle used in the prototype. The purpose of the lifecycle is not to model every possible response action. Its purpose is to ensure that reports remain visible and reviewable after the initial submission, with a status that can be updated as more information becomes available.

2.1.2 Mobile Reporting Constraints

Reporting is expected to happen in the field, so mobile interaction is central. A mobile-first flow prioritizes short forms, simple location entry, and optional evidence upload rather than requiring long text input. VietFlood uses Expo React Native for the citizen and relief clients [10, 11]. To reduce inconsistent spelling of administrative locations, the system uses the Vietnam Provinces Open API as a structured source for provinces and wards [22]. This choice improves filterability and reduces ambiguity when multiple users describe the same area differently.

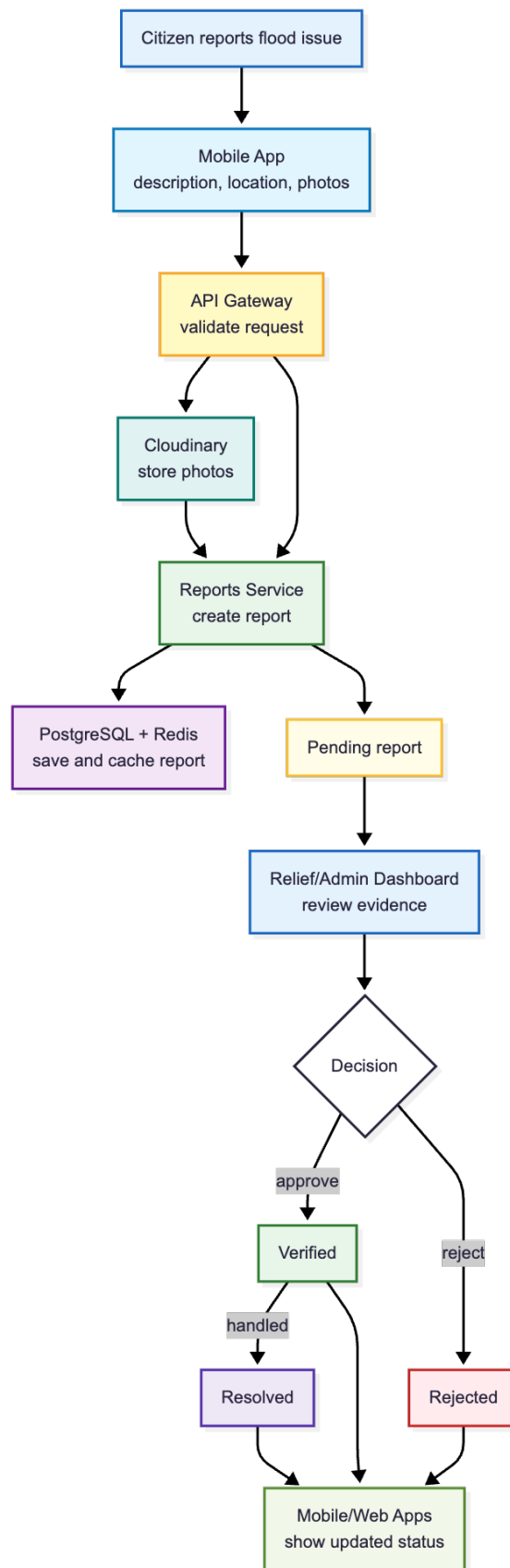


Figure 2.1: VietFlood report flow from mobile submission to administrator review status.

2.2 Research Context and Key Concepts

2.2.1 Citizen Signals in Emergencies

Two streams of related work motivate VietFlood. The first is crowdsourcing and volunteered geographic information in disasters. Goodchild and Glennon describe crowdsourcing geographic information as a research frontier in disaster response, highlighting both its potential value and the need for methods that transform observations into usable information products [1]. The second is crisis communication through social media and messaging. Imran et al. survey methods for processing emergency-related social media messages and emphasize the challenges of noise, duplication, and incomplete context [2]. VietFlood does not attempt automated social media extraction, but it is designed to address the same practical problem: field information arrives quickly and inconsistently, and needs structure to support review.

2.2.2 Responsibility-Driven Architecture

The background for VietFlood’s implementation is software architecture that makes responsibilities explicit. Architectural guidance emphasizes designing system structure around responsibilities and quality attributes, rather than only selecting technologies [23]. For this thesis, the most important responsibilities are: (1) public API handling and access control, (2) authentication and session management, (3) report persistence and status updates, and (4) evidence upload and retrieval. The design should also remain operable as a small prototype without a complex production platform.

2.3 Backend Decomposition Options

A single monolithic backend can be an effective approach for small projects. However, as features accumulate, authentication, file upload handling, report persistence, caching, and real-time communication can become tightly coupled. Microservice architecture is often defined as decomposing a system into services around capabilities, trading increased operational complexity for clearer boundaries and independent evolution [3, 4, 5].

VietFlood adopts a modest service split suitable for a thesis prototype. The system uses an API gateway as the client-facing entry point, and internal services for authentication and report management. Supporting infrastructure (RabbitMQ, Redis, PostgreSQL, Cloudinary) is introduced only when it directly supports a needed responsibility.

2.3.1 Gateway Layer and Service Messaging

The API gateway pattern provides one stable surface for clients even when the backend is split internally. In VietFlood, clients call the gateway for authentication and report routes. The gateway then forwards work to internal services using NestJS microservice patterns [18, 19]. This method reduces coupling between clients and internal service topology.

Service-to-service communication uses RabbitMQ request-response messaging [20]. Using explicit messages makes boundaries visible: authentication logic lives in the auth service, while report storage and status changes live in the reports service. The gateway coordinates requests, enforces access control, and handles evidence upload, but it does not directly own all business logic.

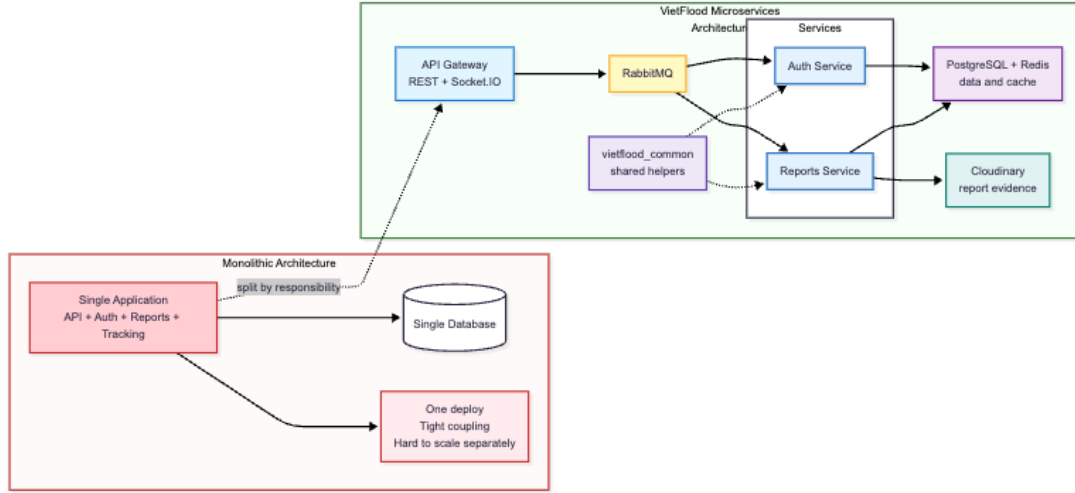


Figure 2.2: High-level contrast between a monolithic backend and the VietFlood service split.

2.4 Storage, Evidence, and Caching Choices

2.4.1 Relational Storage and JSON Evidence Metadata

VietFlood uses PostgreSQL to store users, reports, and evidence metadata. TypeORM maps application entities to relational tables [7]. Evidence items are stored as structured JSON metadata (for example, Cloudinary URL and public ID). PostgreSQL’s JSON support allows this flexible structure while still using a relational database for core report fields such as status and coordinates [8].

2.4.2 External Evidence Storage (Cloudinary)

Storing images as database blobs is not necessary for the thesis prototype and complicates storage and delivery. VietFlood uploads evidence to Cloudinary and stores only references and metadata in the report record [9]. This enables evidence preview in the administrator dashboard while keeping report records small enough to query and update efficiently.

2.4.3 Redis Cache Layer

Redis is used as a cache layer to reduce repeated reads for frequently requested report details [21]. Caching is kept conservative in the prototype. PostgreSQL remains the source of truth, while Redis stores temporary copies that can be refreshed after report updates. This design avoids complex cache invalidation rules while still demonstrating a realistic backend optimization approach.

2.5 Access Control and Real-Time Channels

2.5.1 JWT Tokens and Password Hashing

Role-based access control requires reliable authentication. VietFlood uses bcrypt hashing for passwords [24] and JWT access tokens for stateless verification of identity and role claims [6]. Refresh tokens are used to maintain longer sessions while allowing access tokens to remain short-lived.

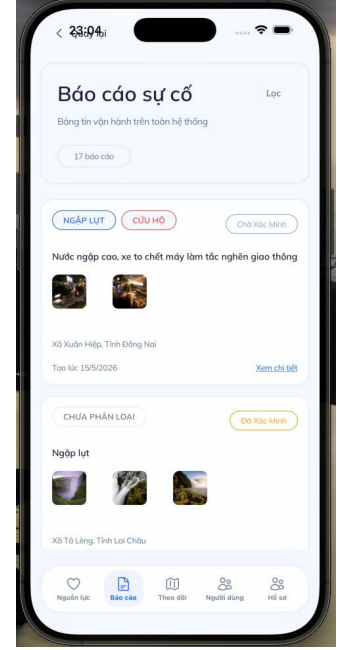
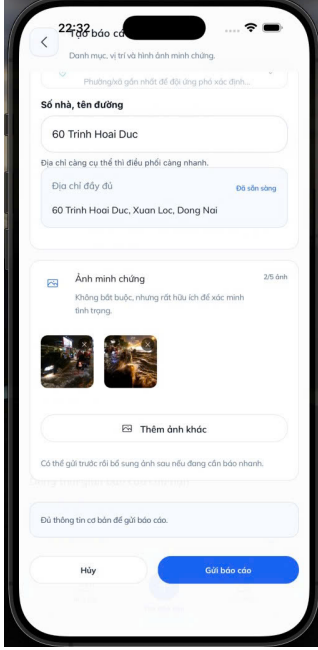


Figure 2.3: Example views of evidence upload on mobile and evidence review on the dashboard.

2.5.2 Socket.IO for Live Location Updates

Real-time communication can support situational awareness when users share location updates. VietFlood includes a Socket.IO gateway for live location broadcasting [25]. The prototype validates coordinate ranges and broadcasts events between connected clients. This provides a minimal foundation for live tracking without expanding scope into persistent tracking histories or assignment workflows.

2.6 Deployment and Environment Repeatability

Repeatable environments are important even for prototypes. VietFlood uses Docker Compose to run backend services and infrastructure dependencies locally [13]. The deployment approach follows standard CI/CD practices: images can be built and deployed automatically using GitHub Actions and Azure App Service container deployment [14, 15]. Continuous delivery guidance motivates keeping build and deployment steps repeatable and automated rather than relying on manual server setup [16, 17].

Chapter 3

Methodology

This chapter explains the design and development method used to build VietFlood. The work follows a build-and-check approach common in software engineering: define scope, model the actors and workflows, design the system structure and data model, implement the prototype, and validate the result through end-to-end scenarios. The goal is not to run a controlled experiment. The goal is to produce a coherent prototype whose structure matches the responsibilities of reporting, review, and role-based access control [23].

3.1 Method Overview

The project targets one specific part of flood response: turning scattered observations into structured reports that can be reviewed and tracked over time. VietFlood does not attempt to predict floods, dispatch resources, or replace official warning systems. The methodology therefore focuses on features that can be implemented and verified within a thesis prototype:

- authentication and session handling;
- report submission with location fields and optional evidence;
- persistent storage of reports and evidence metadata;
- administrator review and status updates;
- relief-oriented reading views, including map-related screens;
- a minimal real-time channel for location sharing.

3.2 Design Inputs and Constraints

Requirements were derived from the intended workflow and the realities of mobile reporting during disruptive conditions. The system must accept short, incomplete, and sometimes noisy input, while still producing records that can be searched and compared. Constraints that shaped the method include limited development time, prototype-level testing, and the lack of validated field datasets.

3.2.1 Functional Goals

Requirements are grouped by the three roles implemented in the prototype:

- **Citizen:** register/sign in, manage profile fields, create reports with address and coordinates, attach photo evidence, and view personal report history.
- **Relief:** browse report information and location-oriented views without changing report status.

- **Administrator:** review all reports, search/filter records, inspect evidence and reporter details, update report status, and manage user accounts.

Each report stores a short description, location fields (province, ward, address line, latitude/longitude), severity/urgency values, timestamps, and a review status. Status values are kept intentionally small in this thesis version: **pending**, **verified**, **resolved**, and **rejected**. This is enough to demonstrate review and follow-up without introducing a complex workflow engine.

3.2.2 Quality and Operational Goals

Several non-functional goals guided design choices:

- **Separation of concerns:** authentication logic should not be mixed with report persistence and evidence handling.
- **Evidence storage outside the database:** images should be stored in a media platform, while the database keeps searchable metadata and report fields.
- **Repeatable runtime:** the system should run locally and in deployment using the same container-based configuration.
- **Simple boundaries:** service splits should be justified by responsibility, not by a desire to maximize the number of services.

3.3 Role and Use-Case Modeling

Actors were modeled first because they define who can perform each operation and what information each role should see. Use-case diagrams were used to keep the prototype scope clear and to avoid designing features that do not fit the stated responsibilities.

3.3.1 Citizen Role

The citizen actor represents a member of the public reporting an observed flood situation. The citizen workflow begins with registration or sign-in, then continues to report creation by entering a description, selecting location fields, attaching photos when available, and submitting the report. The mobile client can help by reading device location and by listing Vietnam administrative divisions to reduce inconsistent spelling.

3.3.2 Relief Role

The relief actor represents users who need situational awareness from submitted reports. In the prototype, the relief role can browse report lists, open details, and use map-related views. The relief role does not verify or reject reports. Keeping verification in the administrator workflow avoids conflicting status changes and keeps the trust signal (status) under one authority in this thesis scope.

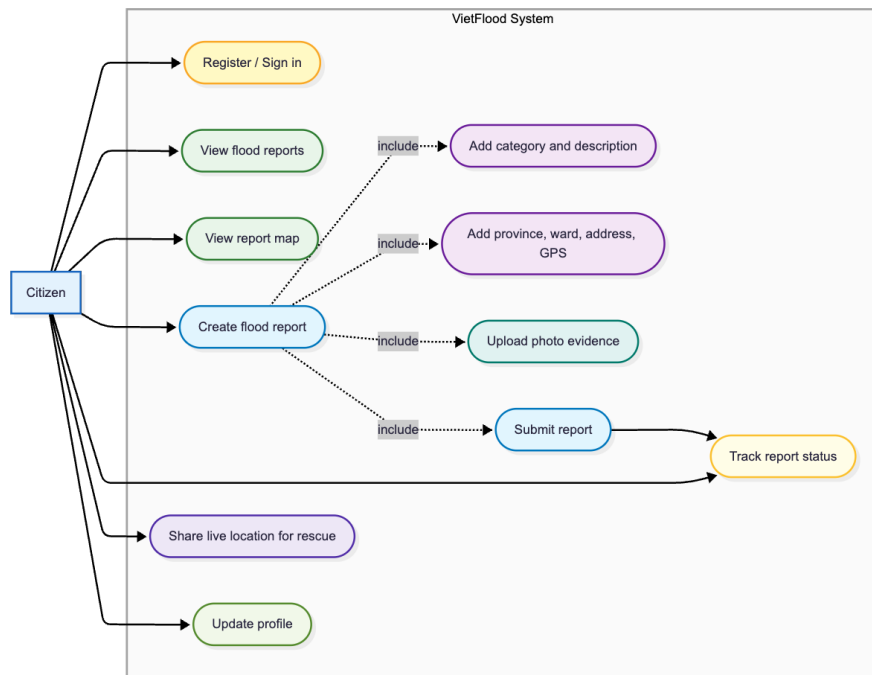


Figure 3.1: Citizen capabilities supported by VietFlood (use case view).

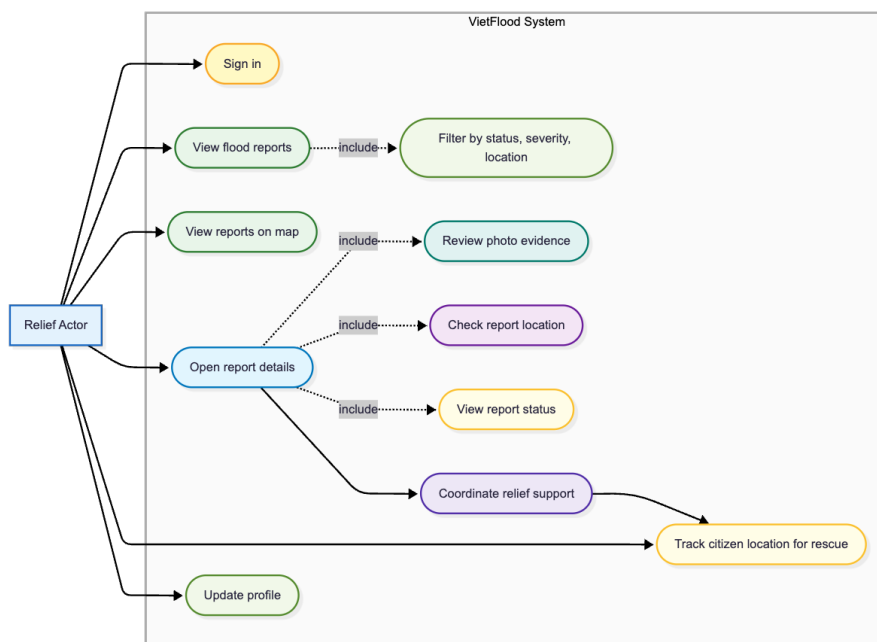


Figure 3.2: Relief browsing features supported by VietFlood (use case view).

3.3.3 Administrator Role

The administrator actor uses the web dashboard to review incoming reports. The dashboard presents report fields, reporter details, evidence previews, and tools for filtering and searching. Administrator actions include changing status values and managing user records. The web client checks the current profile and role before showing administrator screens so a non-admin account cannot access the review workspace by direct URL navigation.

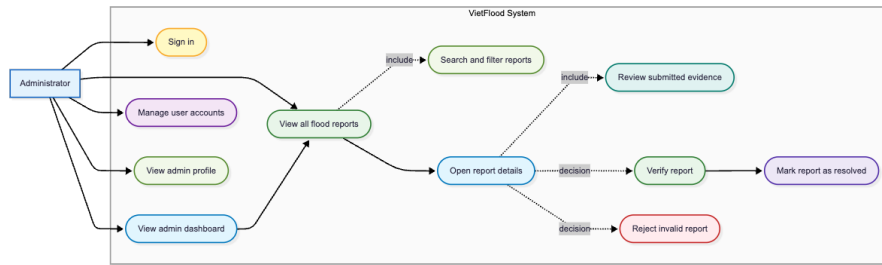


Figure 3.3: Administrator review and management features (use case view).

3.4 Development Materials and Tools

Development work was organized into four code modules to keep responsibilities clear. The backend module contains the NestJS API gateway and internal services [18, 19]. The shared module (`vietflood_common`) provides common infrastructure utilities. The mobile module contains the Expo React Native application [10, 11]. The web module contains the Next.js administrator dashboard [12]. Table 3.1 summarizes the main tools used.

Table 3.1: Materials, tools, and runtime dependencies used during VietFlood development.

Area	Tools and purpose
Backend	NestJS, TypeScript, TypeORM, PostgreSQL, RabbitMQ, Redis, Socket.IO, JWT, bcrypt.
Shared package	<code>vietflood_common</code> for structured logging, Redis client setup, and Cloudinary file operations.
Mobile client	Expo, React Native, TypeScript, React Navigation, Redux Toolkit, image picking, secure token storage, device location, and Vietnam Provinces Open API.
Web dashboard	Next.js, React, TypeScript, Tailwind CSS, protected admin login, token refresh, report overview, search, filtering, evidence preview, and status update.
Runtime infrastructure	Docker, Docker Compose, PostgreSQL, Redis, RabbitMQ, Cloudinary, and Azure App Service configuration.

3.5 Architecture Design Approach

The system was designed as a small set of services rather than a single monolithic backend. The point of the service split is not complexity; it is ownership. Authentication and report persistence evolve differently and have different failure modes. Keeping them separate reduces accidental coupling and keeps code easier to reason about during iteration. At the same time, the number of services is kept low to avoid operational overhead that would not pay off at prototype scale [3, 4, 5].

In VietFlood, the API gateway is the single public entry point. It handles HTTP routes and the Socket.IO channel exposed to clients. The authentication service owns registration, sign-in, token refresh, profile operations, and user management. The reports service owns report creation, listing, status updates, evidence metadata storage, and cache invalidation. Redis, RabbitMQ, PostgreSQL, and Cloudinary provide supporting infrastructure for caching, messaging, persistence, and evidence storage.

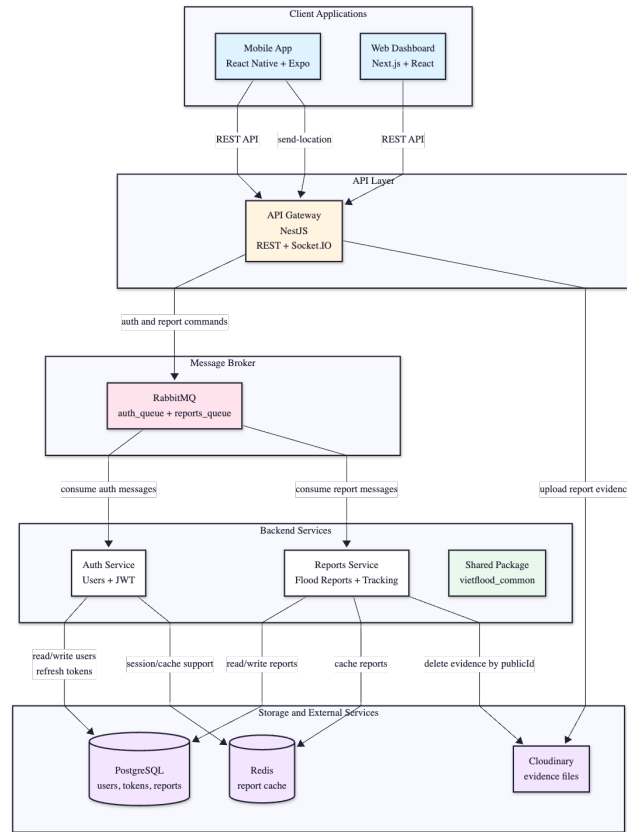


Figure 3.4: System architecture overview of VietFlood components and their interactions.

3.5.1 Gateway Interface Design

The gateway exposes the REST interface consumed by both the mobile and web clients. Authentication routes are grouped under `/auth` and report routes under `/reports`. Report creation is implemented as a multipart upload; the gateway processes uploaded files, uploads evidence to Cloudinary, and then forwards the final report payload to the reports service. Centralizing evidence upload in the gateway avoids duplicating upload logic across services and keeps clients unaware of internal service boundaries.

The gateway also enforces JWT authentication and role checks. Citizen endpoints are scoped to personal report operations, broader report reading is available to relief and

admin accounts, and administrator-only operations are protected separately.

3.5.2 Service Messaging Design

Internal service calls use RabbitMQ request–response messaging [20]. The gateway sends authentication commands to the auth service and report commands to the reports service. This makes the boundary explicit: the gateway does not directly access the database owned by the services, and each service can validate inputs and return controlled responses.

Messaging also makes troubleshooting more concrete during development. If a report creation fails, the failure can be localized to one of a small number of steps: request parsing, evidence upload, message transport, or service-side validation/persistence.

3.6 Data Modeling Approach

The data model was designed to support the report lifecycle while staying small enough to evolve during prototyping. PostgreSQL is used for persistence and TypeORM entities map application objects to relational tables [7]. The core tables are users and reports.

User records store identity fields (email, username, phone), a bcrypt-hashed password, role, basic profile information, and timestamps. Uniqueness constraints on email/username/phone prevent duplicate accounts.

Report records store a textual description, category values, administrative location fields (province, ward, address line), coordinates, urgency/severity values, timestamps, and a status field used for review. Evidence is stored as JSON metadata rather than raw bytes: each item includes a delivery URL and a Cloudinary public ID. This keeps media delivery out of the database while still making the report record self-contained for review [8, 9].

3.6.1 Caching Approach

Redis is used as a cache layer, not as the source of truth [21]. The reports service writes a cached report entry after create/update operations and checks the cache before reading from PostgreSQL on lookups. When a report changes or is deleted, the corresponding cache entry is refreshed or removed. Cache scope is kept narrow on purpose to make invalidation behavior clear in a prototype setting.

3.7 Workflow Specification

Workflows were specified as sequences across the client, gateway, and internal services. The primary workflow is report creation. A citizen enters a description and location fields, optionally selects photo evidence, and submits the report from the mobile app. The gateway authenticates the request, uploads evidence to Cloudinary, and forwards the final payload to the reports service. The reports service persists the report and updates the cache.

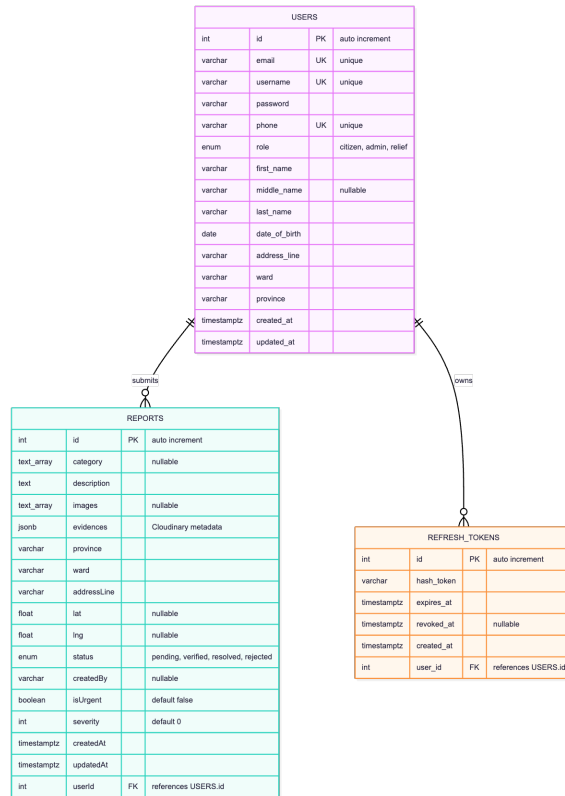


Figure 3.5: Prototype database schema showing the core entities and relationships.

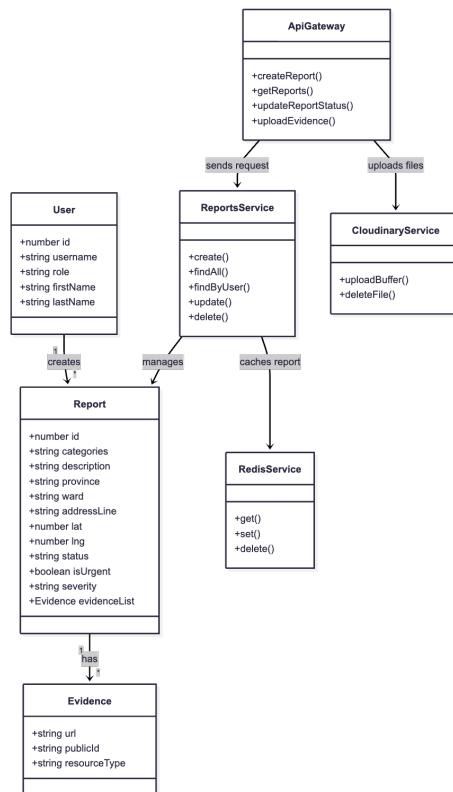


Figure 3.6: Core data structures used in the report workflow.

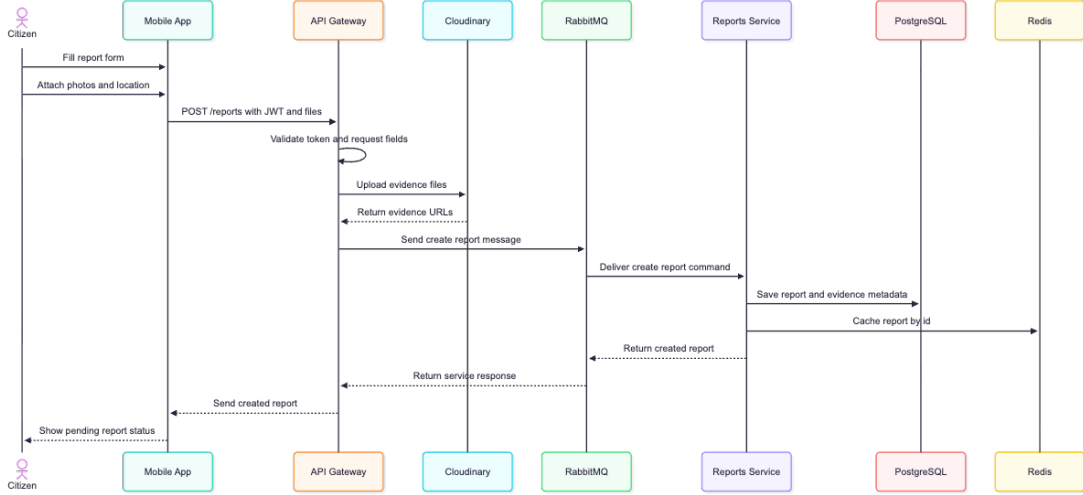


Figure 3.7: Sequence of steps from report submission to persistence and cache update.

3.7.1 Evidence Metadata Handling

Evidence images are uploaded to Cloudinary rather than stored directly in PostgreSQL [9]. The gateway performs the upload and forwards the resulting metadata to the reports service. Storing the Cloudinary public ID alongside the URL makes evidence cleanup possible when reports are deleted. Cloudinary credentials are provided through environment configuration so secrets are not stored in the repository.

3.7.2 Review and Status Updates

Status review is implemented as an administrator-only workflow. Relief users can browse reports, but they do not change status in this thesis version. Administrators update status through the web dashboard; the update request is routed through the gateway to the reports service, which validates the transition, applies the update, and refreshes cached entries.

3.8 Client Development Approach

3.8.1 Mobile Client Design

The mobile client was developed with Expo React Native and TypeScript [10, 11, 26]. The method emphasizes short task completion: sign in, create a report, and view report history. Role-aware navigation is used to present citizen and relief flows without mixing privileged administrator operations into the mobile interface. Redux Toolkit is used to manage authentication state and client-side data flow [27].

Location entry combines two sources. When permission is granted, the device can provide coordinates. Administrative divisions (province/ward) are selected from the Vietnam Provinces Open API to reduce spelling variation and improve consistency across reports [22].

3.8.2 Dashboard Design

The web dashboard was developed with Next.js, React, TypeScript, and Tailwind CSS [12, 28, 26, 29]. It is designed around administrator tasks: triage, evidence inspection,

status updates, and user management. The dashboard signs in through the gateway, stores tokens, refreshes sessions when needed, and checks the loaded profile before rendering administrator-only routes.

After authentication, the dashboard loads report lists through the gateway and presents key fields (status, severity, location summary, description) and evidence thumbnails. Filters and search reduce the time required to find relevant cases when many reports arrive close together.

3.9 Real-Time Channel Approach

VietFlood includes a Socket.IO gateway for live location broadcasting [25]. A client can emit location updates containing latitude/longitude and optional motion fields. The gateway validates coordinate ranges before rebroadcasting updates to connected clients. Invalid payloads receive an error event. This is treated as a prototype capability to demonstrate real-time communication; operational use would require authentication, clearer authorization rules, and persistent history storage.

3.10 Prototype Validation Strategy

Validation combines two checks: (1) responsibility checks at the code level and (2) scenario walkthroughs across the full stack. Docker Compose is used as the repeatable local environment for backend services and infrastructure dependencies [13].

Scenario walkthroughs focus on whether the prototype supports the intended workflows:

- account registration, sign-in, and profile loading;
- report creation with location fields and optional photo evidence;
- evidence upload and metadata storage;
- report listing and detail viewing for citizen, relief, and admin roles;
- administrator status updates and cache refresh behavior;
- live tracking event broadcast and error handling.

These checks are appropriate for a prototype thesis project: they confirm that the system behaves as designed without claiming operational readiness for real disaster response.

Chapter 4

Implementation and Result

This chapter describes the implementation of the VietFlood prototype and the concrete outputs produced by the project. The system was implemented to support a complete reporting loop: users authenticate, citizens submit a report with location and optional evidence, the backend stores and serves that report through a stable API, and administrators review and update report status.

The implementation is delivered as four main artifacts: (1) a backend composed of an API gateway, an authentication service, and a reports service; (2) a mobile application for citizen and relief users; (3) a web dashboard for administrators; and (4) a shared library, `vietflood_common`, that provides reusable infrastructure across services.

4.1 Prototype Deliverables and Technology Stack

TypeScript is used across the backend and both client applications to keep models, validation rules, and data contracts consistent during development. The backend services are built with NestJS [18] and use NestJS microservice support to communicate through a message broker [19]. The mobile application is built with Expo React Native [10, 11], and the web dashboard is built with Next.js [12].

The prototype is designed to run as containers. Docker Compose is used for local development to start the gateway, services, RabbitMQ, and Redis in a repeatable environment [13]. For deployment, container images are deployed to Azure App Service [15], and build/deploy automation is handled through GitHub Actions [14]. Sensitive values (database URL, JWT secrets, Cloudinary credentials, broker credentials) are provided through environment variables rather than stored in source code.

Table 4.1: Implementation technologies used in the VietFlood prototype.

Component	Implementation choices
Backend	NestJS services, TypeORM persistence, RabbitMQ messaging, Redis cache, JWT authentication.
Shared library	<code>vietflood_common</code> for logging, Redis helpers, and Cloudinary upload utilities.
Evidence storage	Cloudinary for media uploads and public delivery URLs.
Mobile application	Expo React Native, secure token storage, image picker, and location services.
Web dashboard	Next.js, role-based routes, report review UI, and admin status updates.
Deployment	Docker Compose for local runtime, GitHub Actions automation, Azure App Service containers.

4.2 Backend Implementation

4.2.1 Service Responsibilities and Routing

The API gateway is the only backend component exposed to clients. It receives HTTP requests (including multipart uploads), validates access using guards, and forwards internal work to the authentication or reports service. Internal communication uses RabbitMQ request–response message patterns [20, 19]. This layout keeps transport and access control in one place, while moving business logic and database operations into dedicated services.

Table 4.2: Responsibility split across backend services in VietFlood.

Service	Main responsibilities
API gateway	Public REST routes, file parsing for uploads, JWT and role guards, and routing requests to internal services.
Auth service	User registration, password validation, token issuance, refresh token rotation, and user management.
Reports service	Report persistence, evidence metadata storage, report listing and filtering, status updates, and cache invalidation.

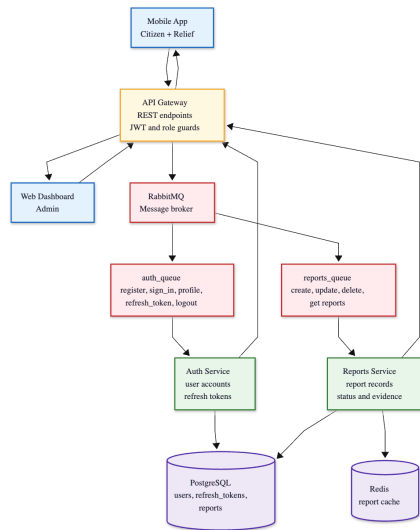


Figure 4.1: RabbitMQ message routing between the API gateway and internal services.

4.2.2 Authentication, Tokens, and Role Checks

VietFlood uses password-based authentication. Passwords are hashed with bcrypt before storage [24]. After login, the authentication service issues an access token and refresh token. Access tokens are JSON Web Tokens that carry identity and role claims needed by the gateway authorization guards [6]. Refresh tokens are stored and rotated to support longer-lived sessions while limiting the impact of token leakage.

The prototype supports three roles: **citizen**, **relief**, and **admin**. Role checks are enforced at the gateway for protected endpoints and admin-only routes. This is a simple model, but it is sufficient to demonstrate the different responsibilities: citizens submit reports, relief users browse and monitor, and administrators review and change status.

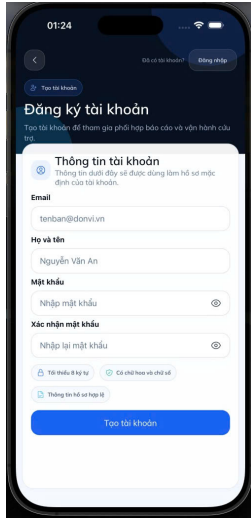


Figure 4.2: Mobile registration view used to create a citizen account.

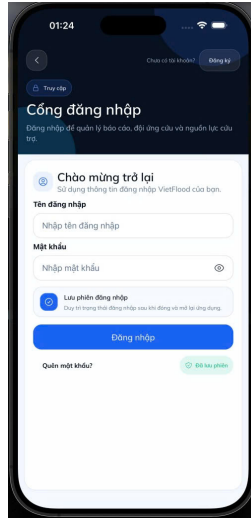


Figure 4.3: Mobile sign-in view that returns access and refresh tokens.

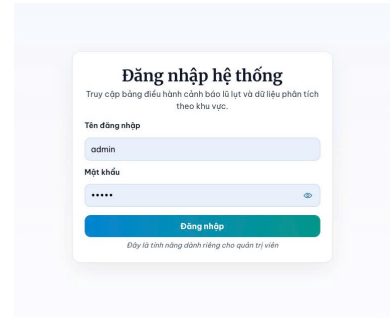


Figure 4.4: Administrator sign-in view in the web dashboard.



Figure 4.5: Refresh token rotation sequence for renewing access tokens and invalidating old refresh tokens.

4.2.3 Report Handling and Evidence Pipeline

Flood reports are created from the mobile client and received by the gateway as multipart form data. When evidence images are present, the gateway uploads files to Cloudinary and produces a metadata list (delivery URL, public ID, and resource type) [9]. The final payload is then sent to the reports service through RabbitMQ. The reports service validates the payload, persists the report using TypeORM [7], and writes a cache entry in Redis for faster follow-up lookups [21].

This evidence-first pipeline keeps the database focused on structured report fields while still making evidence available for review. It also makes the report record self-contained: administrators can review a report by reading one database row plus its evidence metadata, without needing local file storage.

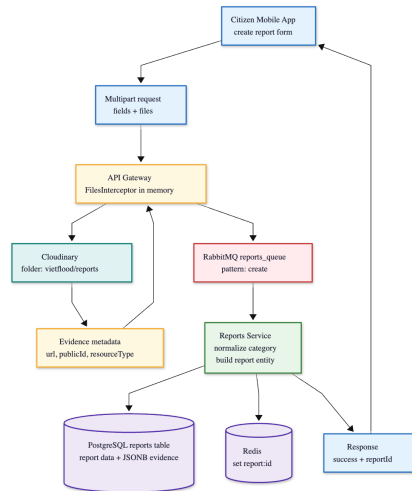


Figure 4.6: End-to-end report handling pipeline from submission to persistence and cache refresh.

Update and delete operations follow the same service boundary. The gateway enforces role permissions, then forwards update/delete commands to the reports service. When a report changes, the reports service invalidates and refreshes cache entries so that subsequent reads reflect the current state.

4.2.4 Shared Infrastructure Utilities

To keep services small and consistent, VietFlood includes a shared library, `vietflood_common`, that exposes a logging module and small wrappers for Redis and Cloudinary. Centralizing these utilities reduces duplicated code and makes it easier to apply consistent error handling and configuration across services.

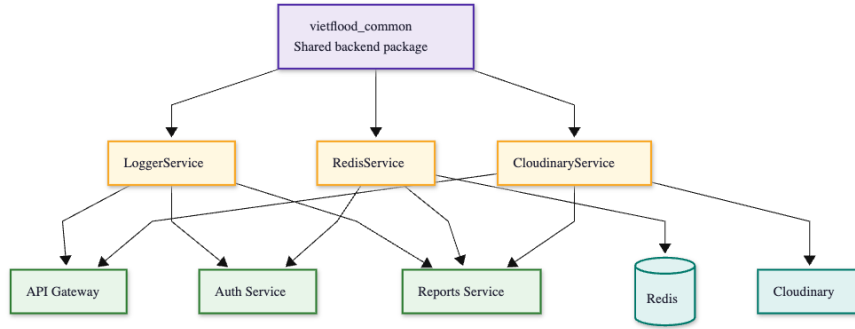


Figure 4.7: Structure of the shared utilities package used by backend services (logging, Redis, Cloudinary).

4.2.5 Live Location Event Channel

The prototype includes a Socket.IO gateway used for live location broadcasting [25]. Clients emit `send-location` events with latitude and longitude values. The gateway validates coordinate ranges and rebroadcasts valid updates to other connected clients using a `receive-location` event. Invalid payloads result in a `location-error` response to the sender. This feature demonstrates real-time event flow and can be extended with authenticated identities and persistence if needed for operational use.

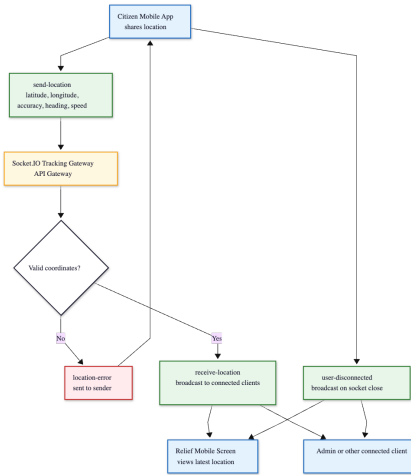


Figure 4.8: Socket.IO location event flow including validation, broadcast, and error responses.

4.3 Client-Side Implementation

4.3.1 Mobile Application Features

The mobile application is the field-facing interface and is designed for quick submission and reading. After sign-in, the app loads the current profile and displays a report list for the user. Citizens can create a report by providing a category, short description, administrative address fields, and coordinates. Evidence images are selected from the device and uploaded as part of the create report request.

The relief role focuses on browsing and map-oriented views. Relief users can open report lists and inspect report details, but they do not change report status in this prototype. That separation avoids conflicting status changes and keeps the verification decision

inside the administrator workflow.

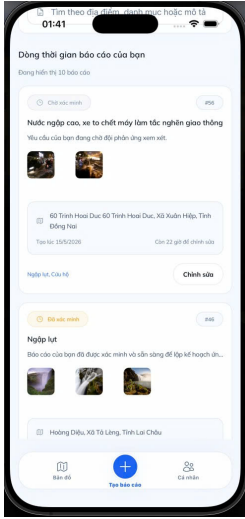


Figure 4.9: Citizen report list showing submissions and current status values.

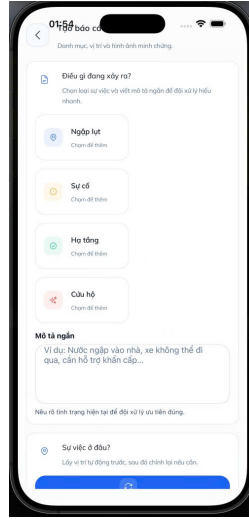


Figure 4.10: Report creation form used to enter description and location fields.

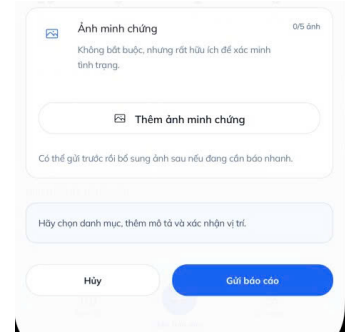


Figure 4.11: Evidence selection and upload workflow on the mobile client.

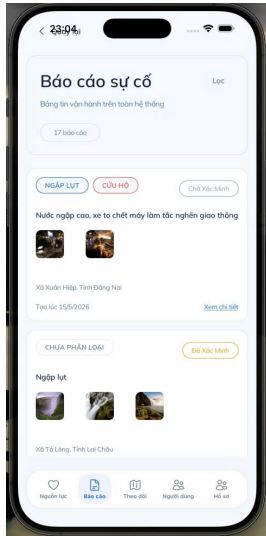


Figure 4.12: Relief list view for browsing broader report records.

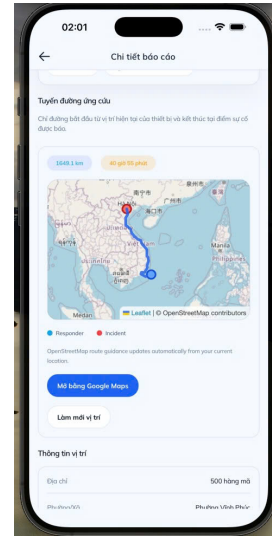


Figure 4.13: Relief map view for locating reports spatially.

4.3.2 Administrator Dashboard Features

The web dashboard is used for report review and user management. After login, it loads the current profile and blocks routes that require administrator access. Administrators can list incoming reports, filter by status, inspect evidence thumbnails, and update status values such as **pending**, **verified**, **resolved**, and **rejected**. Status changes are sent to the gateway and forwarded to the reports service through RabbitMQ so the database and cache remain consistent.

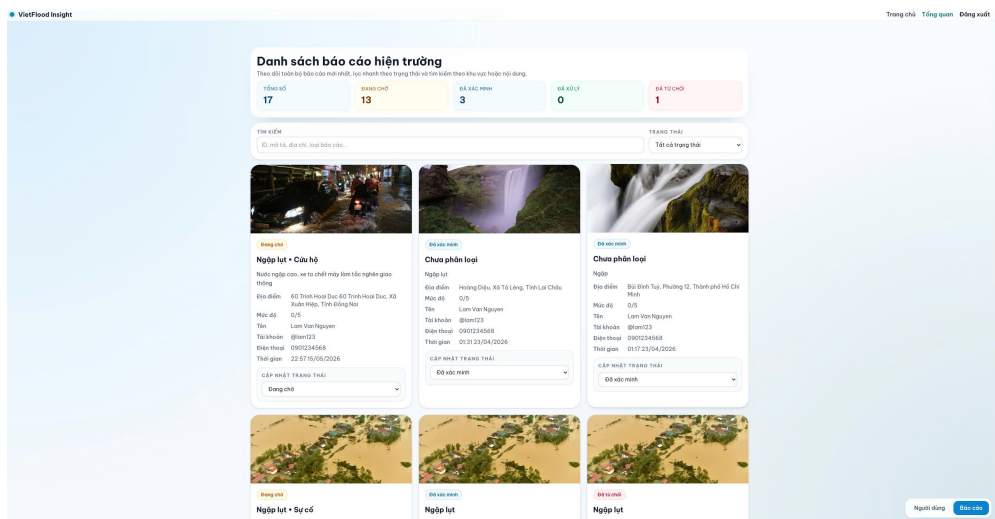


Figure 4.14: Administrator dashboard overview with report cards, filters, and evidence thumbnails.

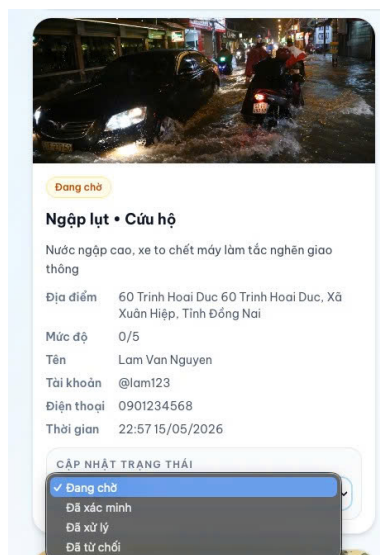


Figure 4.15: Evidence review and status update screen used by administrators.

4.4 Deployment and Release Automation

The prototype runtime is containerized to keep local and deployed environments consistent. Docker Compose is used for local development to start all dependencies (RabbitMQ and Redis) alongside the backend services [13]. Deployment is handled using a simple container-based approach: images are built and published, and Azure App Service is configured to run the latest images with environment configuration provided by the platform [15].

Automation is implemented with GitHub Actions to build images, push them to a registry, and update the deployment configuration [14]. This reduces manual steps and matches continuous delivery practice, where build and deploy logic is kept in version-controlled pipelines [16, 17].

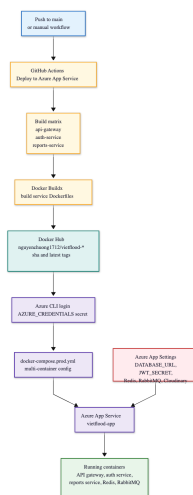
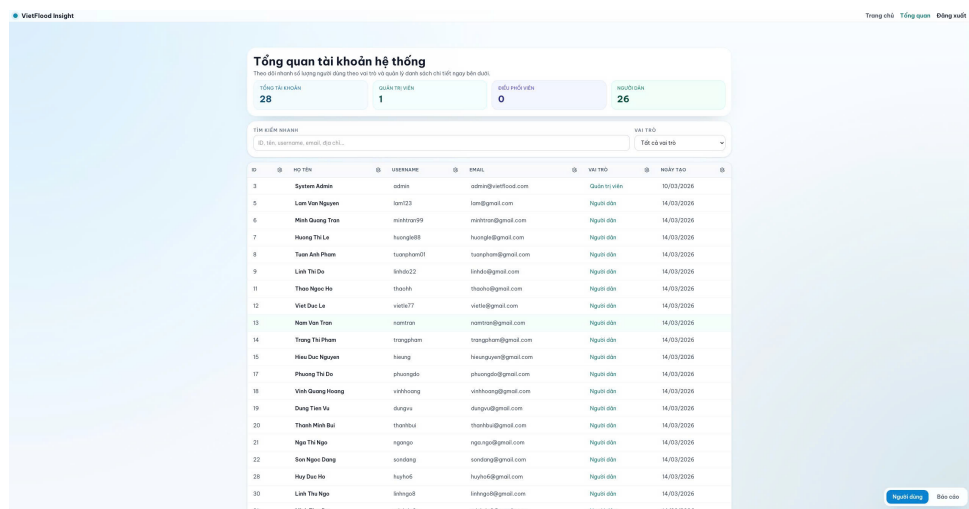


Figure 4.16: CI/CD pipeline from source push to container build, publication, and Azure App Service deployment.

4.5 Result Summary

At the end of implementation, VietFlood can be run locally using Docker Compose and exercised through the mobile and web clients. The backend exposes REST endpoints for authentication and report management through the API gateway. A full endpoint list is summarized in Appendix A (Table A.1), including representative response samples used in the evaluation.

The system demonstrates the intended role split in practice: citizens submit reports, relief users browse location-aware views, and administrators review and update report status. These results are evaluated in Chapter 5 through scenario walkthroughs and functional test checklists.



Chapter 5

Evaluation

This chapter evaluates the VietFlood prototype as an implemented flood reporting system. The evaluation does not attempt to measure real-world impact in a live flood response operation. Instead, it checks whether the delivered backend services and client applications support the intended workflow from Chapter 4: users can authenticate, submit reports with location and evidence, store and retrieve reports, and review them through role-based interfaces.

The evaluation uses scenario walkthroughs and source-level inspection of the API gateway, authentication service, reports service, mobile application, web dashboard, and shared library. A complete list of REST endpoints and representative responses are provided in Appendix A (Table A.1). The system-level test cases used as a checklist for this evaluation are listed in Appendix C (Tables C.1–C.3). The results here should be read as prototype validation rather than production certification.

5.1 Evaluation Approach

The evaluation is structured around the three user roles supported by VietFlood:

Citizen: register and sign in, create flood reports with location and evidence, and view personal report history on mobile.

Relief user: browse report information and location-oriented views on mobile.

Administrator: review reports, inspect evidence, change report status, and manage users on the web dashboard.

On the backend, the API gateway acts as the public interface for HTTP routes and Socket.IO events [25]. Authentication and profile logic are handled by the auth service. Report creation, update, deletion, and cache handling are handled by the reports service. The gateway communicates with internal services through RabbitMQ message patterns [20]. This separation follows the goal of defining clear service responsibilities while keeping the system manageable as a prototype [23].

5.2 Workflow Coverage

The central functional requirement is the end-to-end report workflow. A citizen submits a report from the mobile client. The gateway receives multipart form data, uploads evidence files to Cloudinary [9], and forwards the final payload to the reports service through RabbitMQ [20]. The reports service persists the report in PostgreSQL and stores evidence metadata as JSON fields [8]. After creation and update operations, a Redis cache entry is written to support subsequent lookups [21].

On the review side, administrators use the web dashboard to list reports, inspect evidence thumbnails, and update status values such as **pending**, **verified**, **resolved**, and **rejected**. Relief users can read report information through role-restricted endpoints and present it in list and map oriented views on the mobile application. Together, these features form the expected loop: submission, storage, review, and status update.

Table 5.1: Observed coverage of key VietFlood workflows in the prototype.

Workflow	Primary interface	Observed behavior in prototype
Citizen sign in and profile access	<code>/auth/sign_in</code> , <code>/auth/profile</code>	Tokens are issued after password validation; protected profile data is returned when a JWT is provided.
Create report with evidence	<code>/reports/create</code> (multipart)	Evidence files are uploaded to Cloudinary and stored as metadata; the report is persisted in PostgreSQL and cached in Redis.
View citizen report history	<code>/reports/user</code>	The signed-in citizen can list reports associated with the current account.
Admin review and status update	Web dashboard + protected report routes	Administrators can list reports, inspect evidence, and update report status values through the gateway.
Live tracking broadcast	Socket.IO: <code>send-location</code>	Valid coordinate payloads are broadcast to connected clients; invalid payloads return an error event.

5.3 Client Behavior Review

The mobile application is the main field interface and is implemented with Expo React Native [10, 11]. It supports citizen sign-in, profile viewing, report creation with evidence selection, and report history viewing. Relief features focus on reading and mapping report information rather than creating new reports, which matches the intended division of responsibilities between citizen reporting and relief awareness.

The web dashboard is implemented with Next.js [12] and is evaluated as an administrative workspace. Its main functions are report review, evidence inspection, and status updates. It also loads the current profile and blocks dashboard access when a user does not satisfy role checks.

One weakness observed in the client layer is data contract consistency. The mobile code includes normalization logic for location fields such as `lat/lng` and `latitude/longitude`. This improves tolerance during development, but it indicates that the API contract should be stabilized before broader use, with consistent DTO validation and a single naming convention for coordinates and address fields.

5.4 Storage and Data Consistency

Evidence files are stored in Cloudinary, while PostgreSQL stores report data and evidence metadata. This keeps the database focused on structured fields and avoids storing raw media as database blobs [9]. Evidence metadata is stored in JSON fields (URL, public ID, resource type), which is a reasonable prototype choice when the evidence structure may evolve [8].

The Redis cache is used conservatively. A report cache entry is written after create and update operations, and it is cleared and refreshed when a report changes or is deleted [21]. This design reduces repeated reads for individual report lookups, but it does not optimize large list queries. If VietFlood expands to heavier map queries, province filtering, or large result sets, caching and indexing should be extended beyond single report keys, and pagination should be enforced across list endpoints.

5.5 Access Control and Security Considerations

Route protection is enforced at the API gateway through JWT authentication and role guards. Passwords are stored using bcrypt hashing [24]. JWT access tokens carry identity and role claims used for authorization decisions [6]. Refresh tokens are issued using a

separate secret and expiration policy and are intended to support session control and logout.

Several security concerns remain prototype-level. Refresh token rotation and revocation should be tested under concurrent refresh requests and replay attempts. Socket.IO tracking currently validates coordinate ranges, but the tracking channel should be tied to authenticated identities and a clearer authorization model. Evidence upload should also enforce stricter limits on file size, type, and count to reduce abuse and unexpected storage cost.

5.6 Deployment Considerations

Local deployment uses Docker Compose to run the API gateway, services, RabbitMQ, and Redis in a repeatable environment [13]. The prototype deployment workflow builds Docker images and deploys them to Azure App Service through GitHub Actions [14, 15]. Runtime configuration uses environment variables for database connectivity, RabbitMQ, Redis, JWT secrets, refresh token secret, Cloudinary credentials, and admin seed settings.

The main operational gap is observability. The shared logger provides a consistent logging interface, but a more reliable deployment would also include explicit health endpoints, service-level metrics, and alerting. These are important for detecting failures such as evidence upload errors, queue issues, or service unavailability without manual inspection.

5.7 Summary of Findings

The evaluation shows that VietFlood meets the core prototype requirements. Citizens can submit reports with location and evidence; the backend stores reports and evidence metadata; administrators can review reports and change status; and relief users can browse report information with location context. The main gaps are hardening tasks needed for real deployment: a stable API contract, systematic tests under adverse network conditions, better upload controls, stronger tracking authorization, and monitoring suitable for unattended operation. Within the thesis scope, VietFlood demonstrates that a microservice-based design using NestJS microservice support [19] can provide a practical foundation for multi-platform flood situation reporting.

Chapter 6

Conclusion and Future Works

6.1 Closing Summary

VietFlood was built as a working prototype that turns ad-hoc flood observations into structured reports that can be searched, reviewed, and updated. The project focuses on reporting and coordination support, not flood prediction. A citizen can submit a report from a phone with a location, description, and optional photo evidence, and that report remains available for follow-up rather than disappearing in chat history.

The delivered system includes a NestJS backend split into an API gateway, an authentication service, and a reports service; an Expo React Native mobile application for citizen and relief flows; and a Next.js dashboard for administrators [18, 10, 12]. Reports are persisted in PostgreSQL through TypeORM, evidence is uploaded to Cloudinary and stored as metadata, and report lookups can be accelerated with Redis caching [7, 9, 21]. The gateway uses JWT-based role checks to separate citizen reporting, relief browsing, and administrator review [6]. Service-to-service work is handled using RabbitMQ message patterns to keep HTTP concerns in the gateway and business logic in internal services [20, 19]. This separation follows common microservice design guidance while remaining small enough to deploy as a prototype [3, 4, 5].

The prototype demonstrates the end-to-end reporting loop: create a report, store it with evidence metadata, list and retrieve it through protected endpoints, and update its status after administrator review. The relief role is intentionally read-focused in this version. That decision keeps status changes under administrator control, because status is the main trust signal shown back to other users.

The evaluation in Chapter 5 verifies functional coverage through scenario walkthroughs and inspection of the delivered services and clients. VietFlood still needs field validation under real flood conditions, weak connectivity, and higher traffic before it can be considered for operational use.

6.2 Recommended Next Steps

The next improvements should be driven by real usage feedback on the reporting flow. If users cannot quickly provide the minimum information (location, short description, and one photo), the system will not be useful when conditions are stressful.

Several concrete engineering tasks follow from this prototype:

- **Offline-first reporting:** add local drafts, queued uploads, retry logic, and clear client states for weak networks. Evidence should be compressed on-device, and server-side limits should be enforced to control abuse and storage cost.
- **Data quality and de-duplication:** stabilize DTO naming and validation, and add duplicate detection based on time, distance, and visual similarity to reduce review workload.
- **Review workflow improvements:** expand filters, add clustering views, and support lightweight triage notes so that administrators can process bursts of incoming reports.

- **Tracking hardening:** tie Socket.IO channels to authenticated identities and apply authorization rules that match the intended operational policy.
- **Operations and security:** add health checks, monitoring, and failure alerts; test refresh token rotation under concurrency; and strengthen deployment practices for repeatability and recovery [16, 17, 13].

In a longer project timeline, VietFlood can also be extended beyond reporting: integration with official warning channels, evidence verification partnerships, and map-oriented aggregation for public information. Those extensions should be approached carefully, because public-facing status and location data raises privacy and safety concerns that go beyond the scope of this thesis.

Bibliography

- [1] M. F. Goodchild and J. A. Glennon, “Crowdsourcing geographic information for disaster response: a research frontier,” *International Journal of Digital Earth*, vol. 3, no. 3, pp. 231–241, 2010.
- [2] M. Imran, C. Castillo, F. Diaz, and S. Vieweg, “Processing social media messages in mass emergency: A survey,” *ACM Computing Surveys*, vol. 47, no. 4, pp. 67:1–67:38, 2015.
- [3] M. Fowler and J. Lewis, “Microservices: A definition of this new architectural term.” <https://martinfowler.com/articles/microservices.html>, 2014. Accessed: 2026-05-17.
- [4] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*, pp. 195–216, Springer International Publishing, 2017.
- [5] J. Thönes, “Microservices,” *IEEE Software*, vol. 32, no. 1, p. 116, 2015.
- [6] M. B. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” RFC 7519, Internet Engineering Task Force, 2015. Accessed: 2026-05-17.
- [7] TypeORM, “Typeorm documentation: Getting started.” <https://typeorm.io/docs/getting-started/>, 2026. Accessed: 2026-05-17.
- [8] PostgreSQL Global Development Group, “Postgresql documentation: Json types.” <https://www.postgresql.org/docs/current/datatype-json.html>, 2026. Accessed: 2026-05-17.
- [9] Cloudinary, “Upload api reference.” https://cloudinary.com/documentation/image_upload_api_reference, 2026. Accessed: 2026-05-17.
- [10] Expo, “Expo documentation.” <https://docs.expo.dev/>, 2026. Accessed: 2026-05-17.
- [11] Meta Open Source, “React native documentation: Getting started.” <https://reactnative.dev/docs/getting-started>, 2026. Accessed: 2026-05-17.
- [12] Vercel, “Next.js documentation.” <https://nextjs.org/docs>, 2026. Accessed: 2026-05-17.
- [13] Docker, “Docker compose documentation.” <https://docs.docker.com/compose/>, 2026. Accessed: 2026-05-17.
- [14] GitHub, “Github actions documentation.” <https://docs.github.com/en/actions/>, 2026. Accessed: 2026-05-17.
- [15] Microsoft, “Continuous deployment with custom containers in azure app service.” <https://learn.microsoft.com/en-us/azure/app-service/deploy-ci-cd-custom-container>, 2026. Accessed: 2026-05-17.

- [16] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [17] G. Kim, P. Debois, J. Willis, and J. Humble, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016.
- [18] NestJS, “Nestjs documentation.” <https://docs.nestjs.com/>, 2026. Accessed: 2026-05-17.
- [19] NestJS, “Nestjs microservices documentation.” <https://docs.nestjs.com/microservices/basics>, 2026. Accessed: 2026-05-17.
- [20] RabbitMQ, “Consumer acknowledgements and publisher confirms.” <https://www.rabbitmq.com/docs/3.13/confirm>, 2026. Accessed: 2026-05-17.
- [21] Redis, “Redis documentation.” <https://redis.io/docs/latest/>, 2026. Accessed: 2026-05-17.
- [22] Vietnam Open API, “Vietnam provinces open api.” <https://provinces.open-api.vn/>, 2026. Accessed: 2026-05-17.
- [23] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 4 ed., 2021.
- [24] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *1999 USENIX Annual Technical Conference*, (Monterey, California), USENIX Association, June 1999. Accessed: 2026-05-17.
- [25] Socket.IO, “Socket.io documentation.” <https://socket.io/docs/v4/>, 2026. Accessed: 2026-05-17.
- [26] TypeScript, “Typescript documentation.” <https://www.typescriptlang.org/docs/>, 2026. Accessed: 2026-05-17.
- [27] Redux Toolkit, “Redux toolkit documentation.” <https://redux-toolkit.js.org/>, 2026. Accessed: 2026-05-17.
- [28] React, “React documentation.” <https://react.dev/>, 2026. Accessed: 2026-05-17.
- [29] Tailwind Labs, “Tailwind css documentation.” <https://tailwindcss.com/docs>, 2026. Accessed: 2026-05-17.

Appendix A

API Reference and Sample Payloads

This appendix summarizes the VietFlood API gateway endpoints and provides representative JSON responses. Token strings, user identifiers, and media URLs are masked with placeholders.

A.1 Endpoint Inventory

Table A.1: REST endpoints exposed by the VietFlood API gateway.

Method	Endpoint	Access	Description
Authentication			
POST	/auth/register	Public	Create a new user account.
POST	/auth/sign_in	Public	Authenticate and obtain access and refresh tokens.
POST	/auth/refresh_token	Refresh token	Issue a new access token and rotate the refresh token.
POST	/auth/logout	Public	Logout and revoke refresh token (if provided).
POST	/auth/refresh	Public	Refresh helper endpoint used by the gateway (payload passthrough).
GET	/auth/profile	JWT	Fetch the current user profile.
PUT	/auth/update	JWT	Update the current user's profile.
PUT	/auth/update/user/:id	JWT + Role (admin/relief)	Update a user profile by user id.
DELETE	/auth/delete/:id	JWT + Role (admin)	Delete a user by user id.
GET	/auth/all	JWT + Role (admin/relief)	List all users.
GET	/auth/relief/users	JWT + Role (admin/relief)	List users (alias of /auth/all).
GET	/auth/relief/users/:id	JWT + Role (admin/relief)	Fetch a user by user id.
Reports			
POST	/reports/create	JWT	Create a new flood report (supports evidence file upload).
GET	/reports	JWT + Role (admin/relief)	Fetch all reports for admin/relief review.
GET	/reports/user	JWT + Role (citizen)	Fetch reports created by the current citizen user.
GET	/reports/:id	JWT + Role (admin/relief)	Fetch reports for a specific user id.
PUT	/reports/update/:id	JWT	Update a report owned by the current user (supports evidence upload).
PUT	/reports/update/:id/admin/:userId	JWT + Role (admin)	Update a report for a specific user id (admin workflow).
PUT	/reports/relief/:id/user/:userId	JWT + Role (admin/relief)	Update a report for a specific user id (relief workflow).
DELETE	/reports/:id	JWT	Delete a report owned by the current user.
DELETE	/reports/update/:id/admin/:userId	JWT + Role (admin)	Delete a report for a specific user id (admin workflow).
DELETE	/reports/relief/:id/user/:userId	JWT + Role (admin/relief)	Delete a report for a specific user id (relief workflow).

A.2 Authentication: Sign In

Endpoint: POST /auth/sign_in

Response body:

```
{
  "accessToken": "<JWT_ACCESS_TOKEN>",
  "refresh_token": "<JWT_REFRESH_TOKEN>"
}
```

Listing A.1: Sign In Response

A.3 Authentication: Token Refresh

Endpoint: POST /auth/refresh_token

Response body:

```
{
  "access_token": "<JWT_ACCESS_TOKEN>",
  "refresh_token": "<JWT_REFRESH_TOKEN>"
}
```

Listing A.2: Refresh Token Response

A.4 User: Profile Payload

Endpoint: GET /auth/profile

Response body:

```
{
  "id": 7,
  "email": "<user_email>",
  "username": "<username>",
  "phone": "<phone_number>",
  "role": "citizen",
  "first_name": "<first_name>",
  "middle_name": null,
  "last_name": "<last_name>",
  "date_of_birth": "2000-01-01",
  "address_line": "<address_line>",
  "ward": "<ward>",
  "province": "<province>",
  "created_at": "2026-05-17T00:00:00.000Z",
  "updated_at": "2026-05-17T00:00:00.000Z"
}
```

Listing A.3: User Profile Response

A.5 Reports: Create Payload

Endpoint: POST /reports/create

Response body:

```
{
  "success": true,
  "message": "Create report successfully",
  "reportId": 41,
  "category": ["flood"],
  "evidences": [
    {
      "url": "<CLOUDINARY_URL>",
      "publicId": "<CLOUDINARY_PUBLIC_ID>",
      "resourceType": "image"
    }
  ],
  "images": [
    "<CLOUDINARY_URL>"
  ]
}
```

Listing A.4: Create Report Response

Appendix B

User Interface Snapshots

Key user interface screens are shown here. These images support the main report without repeating the implementation details.

B.1 Mobile and Dashboard Screens

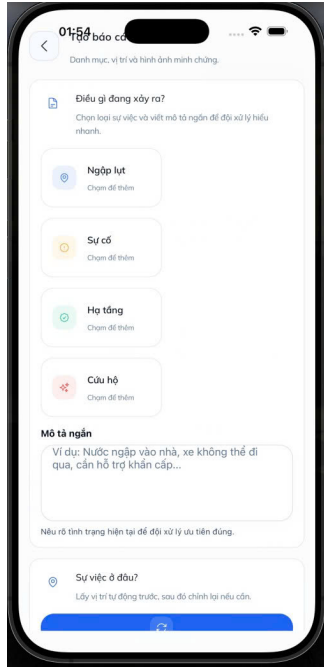


Figure B.1: Mobile report creation screen (citizen workflow).

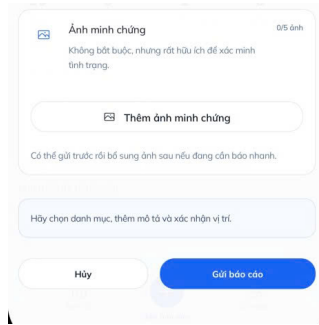


Figure B.2: Mobile evidence selection and upload screen.

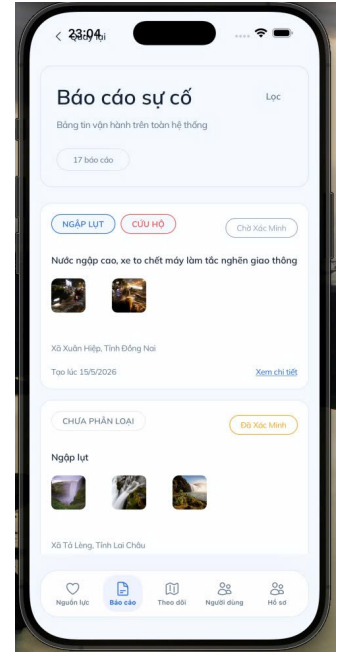


Figure B.3: Relief report list screen for browsing incoming records.

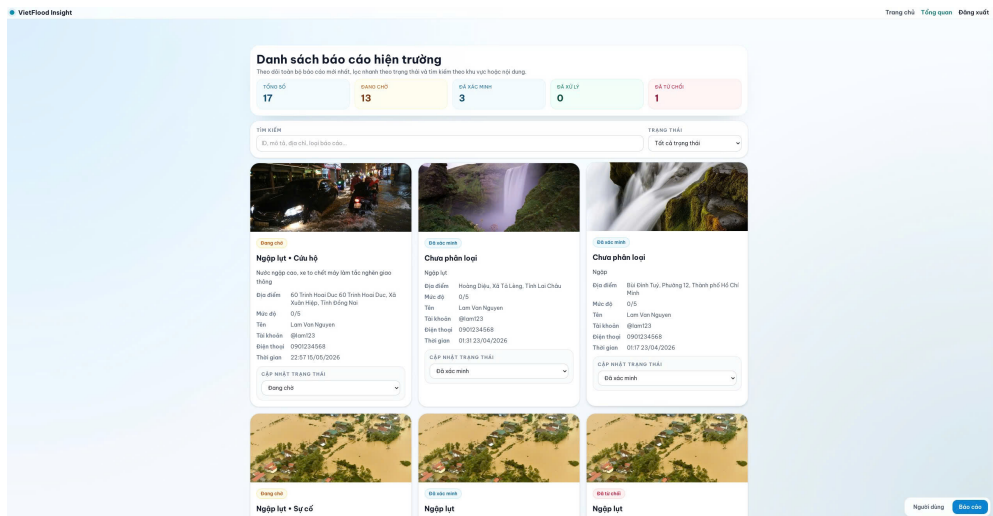


Figure B.4: Administrator dashboard overview with report list and filters.

Appendix C

Prototype Validation Checklist

This appendix lists system-level test cases used to validate the VietFlood prototype. The cases focus on end-to-end behavior through the API gateway, including authentication, role checks, report workflow, and evidence handling.

C.1 Auth and User Scenarios

Table C.1: Authentication and user management validation scenarios.

ID	Feature	Input / Steps (summary)	Expected Result
AUTH-01	Sign in success (POST /auth/sign_in)	Provide valid username and password.	Response contains <code>accessToken</code> and <code>refresh_token</code> .
AUTH-02	Sign in failure	Provide valid username with incorrect password.	Request rejected with an authentication error (no token returned).
AUTH-03	Access protected route without JWT	Call GET /auth/profile without <code>Authorization: Bearer</code>	Request rejected.
AUTH-04	Refresh token success (POST /auth/refresh_token)	Send <code>refresh</code> value in the body containing a valid refresh token.	Response contains <code>access_token</code> and a rotated <code>refresh_token</code> .
AUTH-05	Role enforcement	Call admin/relief-only routes (for example GET /reports) using a citizen JWT.	Request rejected due to insufficient role.
AUTH-06	Admin user listing	Call GET /auth/all with an admin JWT.	Returns list of users with profile fields.

C.2 Report and Evidence Scenarios

Table C.2: Report workflow and evidence handling validation scenarios.

ID	Feature	Input / Steps (summary)	Expected Result
REP-01	Create report with evidence (POST /reports/create)	Submit required fields (<code>description</code> , <code>province</code> , <code>ward</code> , <code>addressLine</code>) with <code>category[]</code> and 1–3 image files.	Response indicates success and returns <code>reportId</code> , <code>evidences</code> metadata, and <code>images[]</code> URLs.
REP-02	Create report without evidence	Submit report payload with no files.	Report created; <code>evidences</code> and <code>images</code> are empty arrays.
REP-03	Citizen fetches own reports (GET /reports/user)	Call as citizen with valid JWT.	Returns list of reports created by the current user.
REP-04	Admin/relief fetches all reports (GET /reports)	Call as admin or relief user.	Returns list of report records for review.
REP-05	Update report by owner (PUT /reports/update/:id)	Update an existing report id owned by the current user; optionally attach additional evidence files.	Returns success message and updated evidence list and image URLs.
REP-06	Delete report by owner (DELETE /reports/:id)	Delete an existing report id owned by the current user.	Returns success confirmation; subsequent fetch does not include the deleted report.

C.3 Tracking Scenarios (Socket.IO)

Table C.3: Socket.IO live tracking validation scenarios.

ID	Event	Input / Steps (summary)	Expected Result
TRK-01	Broadcast valid location (<code>send-location</code>)	Connect a client and emit <code>send-location</code> with valid latitude/longitude ranges.	Server broadcasts <code>receive-location</code> containing the sender id and payload.
TRK-02	Reject invalid payload	Emit <code>send-location</code> with missing or out-of-range coordinates.	Client receives <code>location-error</code> and location is not broadcast.
TRK-03	Disconnect notification	Disconnect a client socket.	Server broadcasts <code>user-disconnected</code> with the socket id.

Appendix D

Runtime Configuration Summary

This chapter summarizes the environment variables used to deploy VietFlood with Docker Compose. Values shown here are variable names and example formats only. Secret values are not included.

Table D.1: Environment variable groups used for VietFlood deployment with Docker Compose.

Group	Example Variables	Notes
Gateway	API_GATEWAY_PORT, CORS_ORIGINS	HTTP port and allowed web origins for the gateway service.
Runtime	NODE_ENV	Runtime mode used by the Node.js applications (for example, production).
Database	DATABASE_URL	PostgreSQL connection string for the deployment database.
Message broker	RABBITMQ_DEFAULT_USER, RABBITMQ_DEFAULT_PASS, RABBITMQ_URL	RabbitMQ credentials and connection URL used by the backend services.
Cache	REDIS_HOST, REDIS_PORT, REDIS_PASSWORD, REDIS_DB	Redis connection details used for caching (host, port, password, database index).
Token signing	JWT_SECRET, REFRESH_SECRET	Secrets used to sign access tokens and refresh tokens.
Admin seed	ADMIN_EMAIL, ADMIN_PASSWORD	Initial administrator credentials used during seed/startup logic.
Cloudinary	CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	Cloudinary credentials used for evidence upload and deletion.

D.1 Operational Notes

- Use masked values for secrets in the thesis and replace them with actual values in deployment.
- Store credentials in CI/CD secrets or environment configuration, not in source control.
- Change default RabbitMQ credentials before production use.
- Set JWT secrets as cryptographically secure strings.